Original Published on 2026-03-21 16:11:30 · 329 reads · 10 · 7 · Copyright 4.0 BY-SA

Article tags: #front end #bootstrap #html

Redis , as the most popular in-memory NoSQL database, has become a standard component in internet projects due to its high-speed read/write speeds and rich functionality—from caching and counters to…distributed Locks are ubiquitous. For beginners, getting started with Redis quickly boils down to mastering two main areas: general commands (the foundation for manipulating all data) and five common data structures (to handle different business scenarios).

Today's blog post will use a "concept + practical instructions" format to guide you step-by-step through understanding these core knowledge points. There are no complicated underlying principles involved, so even beginners can easily follow along and start practicing right away.

First, master the basics: Redis general commands (applicable to all data structures)

General commands are the "basic skills" for operating Redis. Regardless of the data structure being operated on, these commands can be used directly. They are mainly used for key management, data querying, expiration settings, etc. Remembering these can solve 80% of basic operation needs.

The following are the most commonly used and practical general commands, each accompanied by a practical example (assuming we are already connected to a Redis client, the execution effect is explained after the command):

1. Basic key operations (CRUD operations)

- `SET key value`: Set a key-value pair (overwriting an existing key). Example: `SET name zhangsan`→ Add/modify key as name, value as zhangsan
- `GET key`: Retrieves the value of the specified key (returns nil if the key does not exist). Example: `GET name`→ Returns "zhangsan"
- `DEL key [key ...]`: Delete one or more keys (returns the number of keys successfully deleted) Example: `DEL name age`→ If both name and age exist, return 2
- `EXISTS key`Check if the key exists (returns 1 if it exists, 0 if it does not exist). Example: `EXISTS name`→ If 'name' has been deleted, return 0.

2. Expiration time related (high-frequency scenarios)

- `EXPIRE key seconds`Set an expiration time (in seconds) for a specified key. The key will be automatically deleted after the expiration time. Example: `SET code 123456`+ `EXPIRE code 60`→ Verification code 123456 Valid for 60 seconds
- `TTL key`: Check the remaining expiration time of the key (unit: seconds; -1 means never expires, -2 means expired/does not exist). Example: `TTL code`→ If there are 30 seconds left before expiration, return 30.
- `PERSIST key`: Set the expiration time of the cancel button to never expire. Example: `PERSIST code`→ Cancel the expiration time of the verification code and return 1 (success).

3. Other useful general commands

- `KEYS pattern`: Fuzzy match for all keys that match the rules (* matches any character, ? matches a single character). Example: `KEYS n*`→ Matches all keys that begin with 'n' (e.g., name, nickname).
- `TYPE key`: Check the data structure type corresponding to the specified key (returns string, hash, list, etc.) Example: `TYPE name`→ Returns string (because the value of name is a string)
- `FLUSHDB`Clears all keys in the current database (use with caution!). Example: `FLUSHDB`→ After execution, all keys in the current database will be deleted.

Tip: Redis commands are case-insensitive (e.g., SET and set have the same effect), but it is generally recommended to use the same case for keys (e.g., Name and name are two different keys) to avoid confusion.

II. Core Focus: Five Common Redis Data Structures (with Commands and Scenarios)

Redis supports various data structures, among which the five most commonly used and core are: String , Hash, List, Set, and Sorted Set. Each data structure has its own dedicated commands, corresponding to different business scenarios. We will break them down one by one, focusing on remembering the "core commands + application scenarios".

1. String: The most basic and commonly used data structure

Strings are the most basic data structure in Redis. Tl
The maximum size of a single string is 512MB.

It's pretty good, isn't it? One-click collection
VII. Wutong. focus on 10 7 0 share

Dedicated core commands (in addition to general commands):

- `INCR key`Increment the numeric value corresponding to the key by 1 (only applies when the value is an integer). Example: `SET count 10`+ `INCR count`→ count becomes 11 (suitable for use as a counter).
- `DECR key`Decrement the numeric value corresponding to the key by 1. Example: `DECR count`→ count returns to 10.
- `APPEND key value`Example of appending content to the end of an existing string: `SET msg "hello"`+ `APPEND msg " world"`→ msg becomes "hello world"
- `STRLEN key`Example of getting the length of a string: `STRLEN msg`→ Returns 11 (the length of "hello world")

2. Hash : Suitable for storing structured data

A hash structure is similar to a dictionary, where the key is a whole identifier (e.g., user:1001) and the value is a collection of key-value pairs (e.g., name:zhangsan, age:20), making it suitable for storing structured data with multiple attributes .

Core scenario: Storing user datainformation Product details, order information, etc. (no need to split into multiple keys, easy to manage).

Dedicated core commands:

- `HSET key field value`Example of setting a field and value for a hash table: `HSET user:1001 name zhangsan age 20`→ Add name and age for user 1001
- `HGET key field`Example of retrieving the value of a specified field from a hash table: `HGET user:1001 name`→ Returns "zhangsan"
- `HGETALL key`Example of retrieving all fields and values from a hash table: `HGETALL user:1001`→ Returns ["name", "zhangsan", "age", "20"]
- `HDEL key field [field ...]`Example of deleting a specific field from a hash table: `HDEL user:1001 age`→ Delete the age field for user 1001
- `HKEYS key`Example of retrieving all field names from a hash table: `HKEYS user:1001`→ Returns ["name"]

3. List: Ordered, movableRepeated "Queue/Stack"

A list is an ordered collection of strings, elements can be repeated, and it is based on a doubly linked list structure. It supports inserting and deleting elements from the head (left) and tail (right), and the operation speed is extremely fast.

Core scenarios: message queues (such as order notifications), latest message lists (such as Moments updates).stack /Queue implementation.

Dedicated core commands:

- `LPUSH key value [value ...]`Inserting one or more elements from the beginning of a list: Example: `LPUSH msgList "hello"`+ `LPUSH msgList "world"`→ The list becomes ["world", "hello"]
- `RPUSH key value [value ...]`Inserting one or more elements to the end of a list: Example: `RPUSH msgList "redis"`→ The list becomes ["world", "hello", "redis"]
- `LPOP key`: Removes and returns an element from the head of the list. Example: `LPOP msgList`→ Returns "world", the list becomes ["hello", "redis"]
- `RPOP key`Removes and returns an element from the end of the list. Example: `RPOP msgList`→ Returns "redis", the list becomes ["hello"]
- `LRANGE key start stop`: Retrieves a specified range of elements from a list (start starts from 0, stop is -1 to represent all elements). Example: `LRANGE msgList 0 -1`→ Returns ["hello"]

4. Set: An unordered set that does not allow duplicates.

A set is an unordered collection of strings, with no duplicate elements. It is based on a hash table structure and supports set operations such as intersection, union, and difference, making it suitable for deduplication and relationship determination.

Core scenarios: Deduplication of user tags, friend list, mutual friend search, and lottery activities (random element selection).

Dedicated core commands:

- `SADD key member [member ...]`Adds one or more elements to a collection (duplicate elements are automatically removed). Example: `SADD tag:1001 music sports book`→ Add 3 tags to user 1001
- `SMEMBERS key`Example of retrieving all elements from a collection: `SMEMBERS tag:1001`→ Returns ["music", "
- `SREM key member [member ...]`Example of d

It's pretty good, isn't it? One-click collection
VII. Wutong. focus on 10 7 0 share

- `SINTER key1 key2`Find the intersection (common elements) of two sets. Example: `SADD tag:1002 music movie`+ `SINTER tag:1001 tag:1002`→ Returns ["music"]

5. Sorted Set: Ordered, non-repeating, and sorted by "fractions".

Sorted sets are an enhanced version of Sets. Elements cannot be repeated, but each element has a "score". Redis will automatically sort the elements in ascending order based on the score. The underlying structure is a skip list, which balances query and insertion efficiency.

Core scenarios: leaderboards (such as product sales rankings, user points rankings), and weighted message sorting.

Dedicated core commands:

- `ZADD key score member [score member ...]`Add elements to a sorted set (score is the sorting criterion). Example: `ZADD rank:sales 100 apple 80 banana 120 orange`→ Add a new sales ranking list.
- `ZRANGE key start stop [WITHSCORES]`: Retrieves elements within a specified range sorted in ascending order of score (WITHSCORES displays the scores). Example: `ZRANGE rank:sales 0 -1 WITHSCORES`→ Returns ["banana", "80", "apple", "100", "orange", "120"]
- `ZREVRANGE key start stop [WITHSCORES]`Retrieves elements within a specified range sorted in descending order of score (suitable for leaderboards). Example: `ZREVRANGE rank:sales 0 1 WITHSCORES`→ Returns ["orange", "120", "apple", "100"] (top two).
- `ZSCORE key member`Example of getting the score of a specified element: `ZSCORE rank:sales apple`→ Returns "100"
- `ZINCRBY key increment member`: Increase the score of a specified element by a specified value (suitable for updating points and sales). Example: `ZINCRBY rank:sales 10 apple`→ Apple's score becomes 110.

III. Practical Suggestions + Pitfall Avoidance Guide

Mastering the general commands and five major data structures is equivalent to getting started with the core usage of Redis. Here are two practical tips for beginners to help you avoid pitfalls and get started quickly:

1. Try it first, then understand: It is recommended to install a Redis client (such as Redis Desktop Manager), and execute each command one by one according to the command examples in this article. This will allow you to intuitively feel the effect of each command, which is more efficient than rote memorization.
2. Choosing the right data structure is crucial: different scenarios require different data structures. For example, use Hash to store user information and Sorted Set to create leaderboards. Avoid overusing String (for example, splitting multiple user attributes into multiple String keys, which is not conducive to management).
3. Use high-risk commands with caution: `FLUSHDB`(clear the current database), `FLUSHALL`(clear all databases), `DEL *`(delete all keys). These must be disabled in production environments to avoid data loss due to accidental operations.

IV. Summary

Redis's core competitiveness lies in its "high-speed read and write + flexible data structures". To use Redis effectively, the key is to thoroughly understand common commands (master basic operations) and master the five major data structures (match business scenarios).

This article explains the most commonly used and fundamental Redis commands and data structures, covering over 90% of everyday development scenarios. Beginners can first master these concepts before gradually delving into more advanced topics such as persistence (RDB/AOF) and distributed deployment.

去阿里云养一只龙虾，省下一台 MacMini 广告
阿里云轻量应用服务器或无影云电脑上，一键拉起openclaw镜像，直接通过iMessage或钉钉与你的Bot互动

In-depth explanation of Redis 's five core underlying data structures and algorithms qq_36021478's Blog 433
SDS stands for Simple Dynamic String, a completely self-developed string structure by Redis , completely replacing the native C language string. It uses metadata + character arra…

[Redis] Redis Data Types (Part 1) - String Type (Including Common Redis Commands) Be humble in your personal life, be humble in your … 1818
Detailed Explanation of Redis Basic Data Types, Part 1: String Type

In-depth analysis of Redis : Five major data structures and a guide to efficient commands . wxchyy's blog 1585
Redis is a high-performance key-value NoSQL database that supports various data structures such as strings, hashes, lists, sets, a… persistence, high av…

Detailed Analysis of Redis Core Data Structures
Redis is an in-memory database that supports multiple data

It's pretty good, isn't it? One-click collection
VII. Wutong. focus on 10 7 0 share